Real-Time Systems - Laboratory Exercise 6 (week 7)

Lab Note: These t*asks are to be conducted individually (but you should discuss with your friend, don't get lost alone). Although you don't need to submit a report for assessment as in Melbourne, you should make a* ***note document*** *and keep your solution/s* ***code (*.c file/s)****, so that you can review later when doing the final project, as well as prepare for the In-class Tests.*

In the Note document, you should have the required diagrams, screen captures and text to answer the questions. Your note should also contain examples for different input and expected output to demonstrate the behaviour of your program meets the outlined objective. Finally, keep the source files (*.c files) together with the note.

1. QNX Native Interprocess Communication (IPC) - Named Channel Message Passing

QNX implements its own Native message passing mechanism to provide better real-time performance guarantees than are not possible with other network protocols (such as TCP/IP) and buffered message passing such as POSIX mqueues. QNX Native Message Passing differs from the how POSIX mqueues are implemented as they are based on a client-server model that *blocks* in-between the send, receive, acknowledge processes (see lecture notes - L2 and L4). Unfortunately, A major disadvantage with QNX Native Message Passing over POSIX mqueues is that more programming effort is required, although like POSIX message queues, QNX Native Message Passing uses the QNET network protocol to support Interprocess Communication (IPC) services between different nodes on a network.

As QNX Native Message Passing is based on a client-server model, one of your threads is required to act as the server and the other as a client. A thread that wishes to receive messages first creates a channel; another thread that wishes to send a message to that thread must first make a connection by "attaching" to that channel.

In its simplest form this can be done by having the server invoke `name_attach()`, which registers a name under `/dev/name/local` file system on the server node and then waits for a message by calling `MsgReceive()` which effectively blocks the server process/thread until a sends a message. For the client to send a message, it first needs to connect to the communication channel by calling `name_open()`. Then it can send a message by invoking `MsgSend()`. At this point the client blocks and waits for the server to invokes `MsgReply()` to send a reply message back to the client. The function `name_close()` and `name_detach()` are used by the client and server, respectively, to close down the connection.

TASK 1A: Download the NativeMsgPass-Server.c and NativeMsgPass-Client.c files from Canvas and create separate projects for each within the same QNX Momentics workspace. The supplied files shows a simple example of using ***Named*** QNX message passing channels using `name_attach()`, `name_open()`, `MsgReceive()`, `MsgSend()` and `MsgReply()`.

After inspecting the code you will be able to see that the Server and Client code use a #define to identify the location of the attach point.

Server use:

```
#define ATTACH_POINT "myname"
```

Client uses:

```
#define LOCAL_ATTACH_POINT "myname"
```

Or:

```
#define QNET_ATTACH_POINT "/net/ VM_x86_Target01/dev/name/local/myname"
```

The LOCAL_ATTACH_POINT in the client code can be used if both the server and client process are running on the same QNX node, whereas the QNET_ATTACH_POINT is used if you have the client running on another node.

Acknowledgment: Original course notes created by Roy Ferguson, and further developed by Samuel Ippolito from RMIT Melbourne.

Note: If you are using the same QNX nodes as other students, make sure you change the `attach_point` name/s to a unique name in both the server and client code so that unintentional connection to other servers and clients can be avoided.

Note: As any process on the network can create a random message and send it, the first item in the `my_data` and `my_reply` structs in both the server and client code is the `msg_header_t`. See: http://www.qnx.com/developers/docs/7.0.0/#com.qnx.doc.neutrino.lib_ref/topic/n/name_attach.html Typically, the message header struct is constructed out of a `_pulse struct` however, due to the difference in data-type size on 64-bit and 32-bit targets, the header struct in the provided example files has been changed to use a custom header. This allows messages to be passed between nodes with different architecture without the message being corrupted or misaligned.

The usable data item in the `my_data` struct in both the server and client code is the `ClientID` tag (which is completely optional and used for demonstration purposes) and the `data` field. You can use the `ClientID` integer to determine which client sent the message to the server if each client is defined with unique number at compile time. It is declared in the client function at line: `msg.ClientID = 600;`

Also note that some error checking is done in the server code, especially to handle certain automatically sent pulses which follow after a message has been received. A pulse may indicate an error condition or may be a one-way notification of some event such as timer expiry (as you did in Lab 4). You should always include such code in a server. As an example, the current example uses a pulse sent by the client to request that that server process terminates. This request can be ignored by changing the value of the `Stay_alive` variable in the server code while loop.

Note: The lecture examples have far more simplified versions of the server and client code. If you find the provided lab *.c files overwhelming at first, it is suggested you look at the Native Message Passing notes in Lecture 2.

TASK 1B: Once you feel that you understand the sample code, run the server code from Momentics on VM_x86_Target01 and then run 2 separately compiled versions of the client code (using different ClientIDs) on different QNX nodes using the QNET_ATTACH_POINT name that points the correct location on the server Node on the network.

You can transfer the client binaries to the `/tmp (for any other QNX node type)` using the Momentics Target File System Navigator. Use Putty to telnet into the node and execute the client code from the `/tmp` directory. Try starting the clients at the same time, as well as at different times.

Note: The `/fs` directory is a persistent mount on the SD card on some of the supported targets (DE10-nano), however does not allow the permissions to be executable. So, you can only use the /fs directory for text file storage, etc.

In your report, please provide the code snippets showing connection string for each node and indicated the different client IDs used for each client. You should have 3 screenshots, one for the server and each of the clients. You may want to disable the server code from accepting a request to disconnect.

2. QNX Message Passing between threads on the same QNX node

In the previous task, the communication channel was established by using `name_attach()` and `name_open()` functions which provide pathname-to-server-connection mapping. The name channel manages the networking without requiring the server and client process to become resource managers, and is very useful for enabling communication channels between threads

that need to communicate over a network. However, in the case where both server and client threads are running on the same QNX node there are far more computationally efficient methods for establishing a communication channel. This can be achieved by using `ChannelCreate()` on the server, which establishes the communication channel, and `ConnectAttach()` which is used by the client to connect to the channel ID that is owned by the server thread. As long as the client thread is running on the same QNX node and knows the process ID (PID) of the server thread and the open communication channel ID, it is able to connect to the server process and send it messages.

Note: As these communication channels only work between processes running on the same QNX node, the msg_header_t struct in both the server and client code can use the standard `_pulse struct` data-type. That is, you won't need to use the customized headers that were required in the previous task as the channels will not be communicating across different target architectures.

TASK 2A: Download the Connect-MsgPass-Client.c and Connect-MsgPass-Server.c file from Canvas and create separate projects for each within the same QNX Momentics workspace. Instead of using `name_attach()`, `name_open()` to create the communication channel, `ChannelCreate()` and `ConnectAttach()` are used in the server and client threads, respectively. The same `MsgReceive()`, `MsgSend()` and `MsgReply()` functions are used, however the connection is closed with `ConnectDetach()` and `ChannelDestroy()` on the client and server, respectively.See the following link for more information: http://www.qnx.com/developers/docs/7.0.0/#com.qnx.doc.neutrino.lib_ref/topic/m/msgreceive.html

To get the sample code working you first need to start the server process so it displays its PID and channel ID of the communication channel that it opened. Once you know the PID and Channel ID you can then modify the client code to have the correct `serverPID` and `serverCHID` values (both can be found in `main()`). Once you compile the client code with the correct server PID value and channel ID, both threads will start to communicate to each other in a similar fashion to that shown by in Task 1.

Much of the code is the same as the previous example, however the critical difference is that there are no attachment points used. The downside is that the client needs to know the Channel ID and process ID of the server to connect to it, and these numbers will change every time the server process is started.

TASK 2B: Your task is to write some code for server process that stores the servers PID and channel ID in a file at a known location on the server node (e.g. in a file located at: `/fs/myServer.info` or `/tmp/myServer.info`). Then you will need to write some code for the client which is able to automatically retrieve the servers PID and channel ID from the file. You will need to ensure both the server and client are running on same QNX node, however you should be able to demonstrate that the file containing the server information changes each time the server process is invoked.

You may find it helpful to look at the QNX help files to write your file handling procedure: http://www.qnx.com/developers/docs/7.0.0/#com.qnx.doc.neutrino.lib_ref/topic/f/fwrite.html

TASK 2C: Combine your solution for Task 2A (above) with the solution you generated for Task 2D from Laboratory Exercise 4 (Sensor driven Traffic lights using message queues between two processes). However instead of using message queues to deliver the messages between the processes, you should use an unnamed interprocess communication (IPC) channel to pass the sensor data between the processes which are running on the same node. You should design your solution on paper before coding it – you may want to review Lab 4 too.

You may implement the design as you see fit, however an example approach may be to:

1. Turn the `server()` function from Task 2B into a thread and test that both processes work correctly. You may want to convert the `return EXIT_FAILURE;` lines in the server's `if` statements to: `pthread_exit( EXIT_FAILURE );`

2. Turn the `client()` function from Task 2B into a thread and test the new client and server process. You may need to create a struct in the client to hold all the server connection details so it can be passed to the client thread.
3. Confirm everything still works and back it up – important from a sanity point of view.
4. Combine the keyboard reader code from Lab4 Task2D with the client thread and make sure the sensor data gets sent as messages via the communication channel to the server thread. You may want to change the data type from `int` to `char` in the `my_data struct` in both the client and server code. At this point the server thread shouldn't function as a traffic light – just messages with sensor data. Make sure to remove all `sleep()` functions from the client code.
5. Now combine the sensor thread from Lab4 Task2D with the server thread. This will most likely involve passing the sensor data struct to the server thread via a pointer so you can store the received message (ie the sensor data) in a shared variable within the server process (i.e. In a similar fashion to how you passed it to the sensor thread in lab 4).
6. Once you have confirmed that the messages are passing correctly between the client and server processes, and that the server thread is correctly storing the data is `sensor_data` struct within the server process, then you can start adding the state machine loop to the `main()` and the state machine function to the server process.
7. Make sure that you have protected any shared data access using the appropriate mutex primitives in both the server and client processes.

If you follow the steps outlined you should only have 2 threads per process (including `main()`), however there are many ways to solve this problem. In any case, congratulations! You are well on your way to starting the design and implementation of your project work. For your project solution, you may want to consider using named communication channels so you can create communication channels that work over the network between multiple QNX nodes.

In you Note document, you should also take screenshots of the console outputs which contain the servers PID and channel ID as well as the format of the file use to store the IDs. You may also wish to record a step by step procedure as how to run the processes to demonstrate a given set of scenarios.

Acknowledgment: Original course notes created by Roy Ferguson, and further developed by Samuel Ippolito from RMIT Melbourne.